Cloud-Native Microservices-Based Web Application
Project Architecture & Explanation

1. Project Overview

This project is a cloud-native web application designed using modern enterprise technologies. The primary objective of this project is to demonstrate real-world software development practices including microservices architecture, secure authentication, CI/CD automation, containerization, and UI testing within a limited development timeline.

2. Problem Statement & Objective

Traditional monolithic applications are hard to scale, test, and deploy independently. This project solves that problem by breaking the backend into multiple independent microservices, each responsible for a single business capability. The system is secure, scalable, and automation-ready.

3. High-Level Architecture

The application follows a layered architecture consisting of a frontend layer, backend microservices layer, database layer, and DevOps automation layer. Communication between components happens using REST APIs over HTTP/HTTPS.

Browser | React Frontend | REST APIs | ---------- | Auth Service | Core Service | | (Spring Boot) | (Spring Boot) | ---------- | Audit Service (MongoDB)

4. Microservices Breakdown

Auth Service: Responsible for user authentication, registration, JWT token generation, and role-based authorization using Spring Security and MySQL.

Core Business Service: Handles domain-specific business logic such as CRUD operations and application workflows. Uses Hibernate and MySQL for persistence.

Audit / Logging Service: Captures user activity and system logs. MongoDB is used due to its flexible schema and suitability for log data.

5. Security Implementation

Security is implemented using JWT-based authentication. Once a user logs in successfully, a token is generated and sent to the client. This token must be included in every API request. Spring Security validates the token before allowing access to protected resources.

6. CI/CD, Testing & DevOps

Jenkins is used to automate the CI/CD pipeline. On every code push, Jenkins builds the application, executes Selenium-based UI tests, and creates Docker images for deployment. Selenium ensures that critical UI flows such as login and form submission work correctly.

7. Containerization with Docker

Each microservice runs inside its own Docker container, making the application portable and deployment-ready. Containers ensure consistency across development, testing, and production environments.

8. Interview & Placement Value

This project demonstrates strong understanding of backend development, microservices design, security best practices, database integration, frontend-backend communication, automated testing, and CI/CD pipelines. It closely resembles real-world enterprise systems used in the industry.

9. Conclusion

The project is intentionally kept simple in business logic but strong in architectural and technological implementation. This makes it ideal for interviews and placements, where understanding and explanation of concepts matter more than idea complexity.